

GIRL IN BLOOM GLOBAL AMBASSADORS 2026

Eco Girls Collective Operations Guide

How the website and dashboard fit together, how to put them live, how staff use them, and how to fix them when something breaks.

Frontend and backend, verified at handover

26 September 2026. Covers the public website, the staff dashboard, and the API.

Contents

1. [What this system is](#)
2. [Before you go live: checklist](#)
3. [Setup and configuration](#)
4. [Deploying to production](#)
5. [Security model](#)
6. [Staff guide: using the dashboard](#)
7. [The public website](#)
8. [API reference](#)
9. [Database](#)
10. [Testing and verification](#)
11. [Troubleshooting](#)
12. [Known limitations and next steps](#)

1. What this system is

Two applications that work together, each in its own Git repository:

Part	Folder	Repository	Technology	Default port
Frontend (public website + staff dashboard)	frontend/	Aubierge-codes/She-leads-front	Next.js 16, React 19, Tailwind 4	3000
Backend (API + database access)	backend/	Aubierge-codes/She-leads-back	NestJS 11, Prisma 7, PostgreSQL	4000

```
Visitors --> Public website (Next.js) --+
                                         |
                                         +--> API (NestJS) --> PostgreSQL
                                         |
Staff ----> Login + Dashboard (Next.js) --+
```

Staff sign in with email and password. The API returns a login token (JWT) which the dashboard sends with every request.

Two audiences, one site:

- **The public** sees the landing page (/) and donation page (/donate). They can subscribe to the newsletter, send a partnership inquiry, and pledge a donation. They never log in.
- **Staff** sign in at /login and use the dashboard (/dashboard) to manage participants, schools, communities, cleanup events, waste records, inventory, clubs, weekly reports, donations, partnership inquiries and newsletter subscribers.

The frontend never talks to the database. Everything goes through the backend API.

2. Before you go live: checklist

Work through this top to bottom. Items marked **BLOCKER** should not be skipped.

Security

- ☐ **BLOCKER: Change the admin password.** The first admin account was created with a default password (`ChangeMe123!`) that is visible in the public Git history. Sign in, open **Settings** → **Change password**.
- ☐ **BLOCKER: Remove the leftover admin** `admin@sheleads.org` . The original version of the seed script created this account; a later rebrand switched to `admin@ecogirlscollective.org` without removing it, so a database that ran both seeds has **two admins, both with the default password**. Sign in as the Eco Girls admin, open **Settings** → **Team members** and click **Remove** next to `admin@sheleads.org` (unless it genuinely belongs to someone on your team, in which case have them change their password). *This was confirmed on the development database; check your production database the same way.*
- ☐ **BLOCKER: Set a strong** `JWT_SECRET` . The backend refuses to start in production with a missing, placeholder, or short (< 32 character) secret. Generate one: `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`
- ☐ Set `NODE_ENV=production` on the backend. Without it the safety checks are skipped.
- ☐ Set `FRONTEND_URL` to the real public website address so only that site can call the API from a browser.
- ☐ Serve both apps over **HTTPS** (most hosts do this automatically). Login tokens travel in request headers and must not be sent over plain HTTP.
- ☐ If the API sits behind a hosting platform's proxy, set `TRUST_PROXY=1` so the login rate limit works per person.
- ☐ Consider a shorter `JWT_EXPIRES_IN` (for example `12h`) so a lost laptop stays signed in for less time.

Data and operations

- ☐ Run `npx prisma migrate deploy` against the production database (see [Deploying](#)).
- ☐ Turn on **automatic database backups** at your database provider, and test a restore once (see [Database](#)).
- ☐ Set `NEXT_PUBLIC_API_URL` on the frontend to the production API address **before building** it.
- ☐ Run the smoke test against a staging copy ([Testing](#)).

Content (a person must do this)

The public website still shows placeholder text in these places. It is real-people content that only the organization can supply. See [Section 7](#) for exactly where to edit.

- ☐ **BLOCKER (content):** Three Stories cards show `[Add a real quote from ...]` and `[Add school/community]` .
- ☐ **BLOCKER (content):** Nine "Girls in Action" entries show `[Add a real story about this activity]` .
- ☐ Replace stand-in illustrations with real photos when available (Programs and Girls in Action). Two program cards (Community Cleanups, Waste Management) still show a dashed "Placeholder" box.

3. Setup and configuration

Requirements

- Node.js 22 (developed on 22.20) and npm
- PostgreSQL 16
- Git

Backend environment variables (`backend/.env`)

Copy `backend/.env.example` to `backend/.env` and fill it in. On a hosting platform, set the same names in its environment settings instead of committing a `.env` file (the file is git-ignored on purpose).

Variable	Required	Purpose
<code>DATABASE_URL</code>	yes	PostgreSQL connection string, e.g. <code>postgresql://user:pass@host:5432/dbname</code>
<code>PORT</code>	no (3000 if unset)	Port the API listens on. The frontend expects 4000 locally.
<code>NODE_ENV</code>	production: yes	<code>production</code> turns on the safety checks below.
<code>JWT_SECRET</code>	yes	Signs login tokens. 32+ random characters. Changing it signs everyone out.
<code>JWT_EXPIRES_IN</code>	no (<code>1d</code>)	How long a login lasts (<code>12h</code> , <code>1d</code> , <code>7d</code> ...).
<code>FRONTEND_URL</code>	production: yes	Allowed website origin(s) for browsers, comma-separated, no trailing slash.
<code>LOGIN_RATE_LIMIT</code>	no (<code>10</code>)	Login attempts per minute per IP address.
<code>TRUST_PROXY</code>	behind a proxy: <code>1</code>	Use the real client IP from the proxy's <code>X-Forwarded-For</code> .
<code>ENABLE_SWAGGER</code>	no	<code>true</code> shows the API docs page at <code>/api</code> in production. Always on in development.
<code>SEED_ADMIN_EMAIL</code>	no	Email of the first admin (default <code>admin@ecogirlscollective.org</code>).
<code>SEED_ADMIN_PASSWORD</code>	production: yes, first time	Password for the first admin. Also the way to reset a forgotten admin password (see Troubleshooting).

Production safety checks built into the backend

- Refuses to start if `JWT_SECRET` is missing, is the placeholder, or is under 32 characters.
- Logs a warning if `FRONTEND_URL` is not set (it then accepts requests from any website).
- The seed script refuses to create the first admin without `SEED_ADMIN_PASSWORD` .

Frontend environment variable (`frontend/.env.local`)

Variable	Purpose
<code>NEXT_PUBLIC_API_URL</code>	Public address of the backend API (default <code>http://localhost:4000</code>). It is baked into the website at build time , so change it <i>before</i> <code>npm run build</code> , and rebuild if it ever changes.

Run it locally

```
# 1. Backend
cd backend
cp .env.example .env          # then edit DATABASE_URL etc.
npm install
npx prisma migrate deploy     # creates the tables
npm run seed                  # creates the first admin (dev default password: ChangeMe123!)
npm run start:dev             # API on http://localhost:4000

# 2. Frontend (new terminal)
cd frontend
cp .env.example .env.local
npm install
npm run dev                   # site on http://localhost:3000
```

Sign in at `http://localhost:3000/login` with the seeded admin.

4. Deploying to production

The exact clicks depend on your hosting provider; the steps do not.

Backend

```
npm ci                # install ALL dependencies (the build, migrate and seed steps
need the dev tools)
npm run build          # generates the database client and compiles to dist/
npx prisma migrate deploy # applies database migrations (safe: only applies what is
missing)
npm run seed           # first deploy: creates the admin. Later deploys: leaves the
password alone.
npm run start:prod     # node dist/src/main.js
```

Set the [environment variables](#) first. `package.json` also has a `postdeploy` script (`npx prisma db seed`) for hosts that run one automatically.

Order matters: migrate **before** starting the new version, so the code never runs against an out-of-date database.

Use `migrate deploy` in production, never `migrate dev` (which is for development and can reset data).

Frontend

```
npm ci
npm run build          # NEXT_PUBLIC_API_URL must already be set
npm run start          # serves on port 3000 (or your host's PORT)
```

Hosts that build Next.js natively (Vercel, Netlify, and similar) only need `NEXT_PUBLIC_API_URL` set in their environment settings.

After deploying: verify

1. Open the API root URL: it should return `200` with a short greeting.
2. Open the website: the landing page should show real numbers (not dashes) under the headline.
3. Visit `/dashboard` while signed out: you must be redirected to `/login`.
4. Sign in, open **Settings**, change the admin password.
5. Optionally run the [smoke test](#) against the live API using a staging account.

Rolling back

Redeploy the previous version of the code. The migrations so far only add tables and columns, so an older backend keeps working with a newer schema. If a bad migration must be undone, restore the database from a backup (see [Database](#)).

5. Security model

What is public and what needs login

The backend blocks everything by default. Only the routes below are open to the internet.

Open route	Why it is public
GET /	Health check
GET /public/summary	Landing page headline numbers (4 fields only: participants, schools, communities, kg of waste)
GET /public/reach	"Where we work" list (names and states only)
GET /public/impact	Landing page charts (waste by type, participants by status)
POST /auth/login	Sign in (rate-limited)
POST /newsletter	Newsletter signup form
POST /partnerships	"Become a Partner" form
POST /donations	Donation pledge form

Everything else needs a valid login token, including all of the dashboard's data and the **lists** of newsletter subscribers, partnership inquiries and donors. Submitting a form is public; *reading* what people submitted is not.

The public pages deliberately use the trimmed `/public/*` routes rather than the full dashboard routes, so the public website can never expose staff-only fields.

Roles

Role	Can do
ADMIN	Everything a manager can, plus manage team members (Settings → Team members)
MANAGER	Use every dashboard page; cannot see or change who has access

How login works

1. Staff sign in with email and password. Passwords are stored only as bcrypt hashes and are never returned by any API.
2. The API returns a signed token (JWT) valid for `JWT_EXPIRES_IN`. The browser keeps it and sends it with every request.
3. On every request the backend re-checks that the user still exists and is active, so **removing a team member takes effect immediately**, even if they still hold an unexpired token.
4. **Logout** clears the token in that browser. The dashboard also signs the user out automatically if the API answers "401 unauthorized".

Protections in place

Protection	Detail
Brute-force limit	10 login attempts per minute per IP (<code>LOGIN_RATE_LIMIT</code>); then <code>429 Too Many Requests</code> , even for a correct password, until the minute passes
Startup checks	Production will not boot with a weak <code>JWT_SECRET</code>
CORS allow-list	In production only <code>FRONTEND_URL</code> origins are accepted from browsers

Protection	Detail
Input validation	Unknown or malformed fields are rejected with <code>400</code>
Secrets stay out of Git	<code>.env</code> files are ignored; <code>.env.example</code> documents each variable
No secrets in API output	Password hashes are stripped from user lists and report data

Things that are true, but worth knowing

- The token is stored in the browser's `localStorage`. This is common and simple, but it means a cross-site-scripting bug could steal it. The site does not render user-supplied HTML, which limits this risk.
- Changing your password does **not** sign out other browsers already holding a token; those expire on their own after `JWT_EXPIRES_IN`. Removing the user does cut them off immediately.

6. Staff guide: using the dashboard

Signing in and out

Go to `/login`, enter your email and password. **Logout** is in the top-right of every dashboard page. Only people an administrator has added can sign in; there is no public sign-up.

Changing your password

Settings → **Change password**. Enter your current password and a new one (8+ characters) twice.

Adding or removing staff (administrators only)

Settings → **Team members**.

- **Add:** enter name, email, a temporary password (8+ characters) and a role, then **Add team member**. Tell the person their temporary password privately and ask them to change it after first sign-in.
- **Remove:** click **Remove** on their row. They can no longer sign in, and any active session stops working immediately. You cannot remove yourself.

The dashboard pages

Page	What it is for
Dashboard	Overview counts, waste by type, recent activity
Participants	The girls in the programme: add, edit, search, and track status (Active / Inactive / Graduated)
Schools, Communities	Partner schools and communities
Events	Cleanup events. Open an event's Manage panel to record who attended and to enter the waste collected (type, weight, bags). This is where waste figures come from.
Waste Tracking	Read-only overview of waste collected, by type, with the latest records (enter new waste from an Event)
Inventory	Supplies, stock levels, restock/usage transactions, low-stock warnings
Env Clubs	Environmental clubs per school
Reports	Weekly reports; approve or reject submitted ones
Donations	Pledges from the public donation page; update status (Pending → Completed...)
Partnerships	Inquiries from "Become a Partner"; open one to read it, then mark New → Contacted → Closed
Newsletter	Everyone who signed up; Copy all emails puts the list on your clipboard to paste into an email tool

Deleting records. "Delete" hides a record everywhere (lists, counts, charts, the public website's numbers) but keeps it in the database, so nothing is lost by accident. Restoring is currently done through the API (`POST /<resource>/:id/restore`), not a button.

Who did what. Weekly reports and inventory transactions record the signed-in person who created them.

7. The public website

Where its numbers come from

The landing page reads live data from the API through the open `/public/*` routes, so the numbers update as staff record data. If the API is unreachable the number areas show `–` instead of breaking.

Forms

Form	What happens
Footer Subscribe	Stores the email in the newsletter list (repeat signups are ignored). Staff see it under Dashboard → Newsletter .
Become a Partner	Stores the inquiry. Staff see it under Dashboard → Partnerships .
Donate	Records a pledge with status <i>Pending</i> . No payment is taken (see limitations). Staff follow up and mark it <i>Completed</i> .

Content that still needs real information

These are edited in code today. Change the text, save, rebuild and redeploy the frontend.

What	Where
Three Stories: quote and school/community	<code>frontend/app/page.tsx</code> (search for <code>Add a real quote</code>). The portrait photos are already real.
Nine "Girls in Action" stories	<code>frontend/data/girls.ts</code> , the <code>description</code> of each entry (search for <code>Add a real story</code>). Add a real <code>name</code> too, if the person has consented to being named.
Program and story photos	Put images in <code>frontend/public/images/</code> and set <code>photoSrc</code> in <code>frontend/data/girls.ts</code> , or <code>illustrationSrc</code> for the programs in <code>frontend/app/page.tsx</code> . Entries without one show a placeholder box.
Hero numbers (<code>80+</code> girls, <code>3+</code> schools) and the headline strip	<code>frontend/app/page.tsx</code> . Some figures are typed in by hand rather than read live, so review them.

Please only publish quotes and names of participants (many are young people) with their, and a guardian's, consent.

Files worth knowing in the frontend

Path	Purpose
<code>app/page.tsx</code>	The landing page
<code>app/donate/page.tsx</code>	Donation flow
<code>app/login/page.tsx</code>	Staff sign-in
<code>app/dashboard/layout.tsx</code>	Sidebar, top bar, sign-in guard, Logout
<code>app/dashboard/*/page.tsx</code>	One file per dashboard page
<code>lib/api.ts</code>	Every API call and response type; attaches the login token
<code>lib/auth.ts</code> , <code>lib/use-session.ts</code>	Login token storage and the session hook
<code>components/</code>	Shared pieces (modal, cards, charts, illustrations)

8. API reference

Base URL: the backend address (locally `http://localhost:4000`). Send `Authorization: Bearer <token>` on every route not marked **Public**. Errors return JSON like `{ "statusCode": 400, "message": "..." }`.

Area	Routes	Access
Health	GET <code>/</code>	Public
Public site data	GET <code>/public/summary</code> , <code>/public/reach</code> , <code>/public/impact</code>	Public
Auth	POST <code>/auth/login</code>	Public
	PATCH <code>/auth/password</code> (<code>currentPassword</code> , <code>newPassword</code>)	Login
Users	GET/POST <code>/users</code> , GET/PATCH/DELETE <code>/users/:id</code> , POST <code>/users/:id/restore</code>	Admin
Dashboard	GET <code>/dashboard/summary</code> , <code>/dashboard/recent-activity</code>	Login
Participants	GET/POST <code>/participants</code> , GET/PATCH/DELETE <code>/participants/:id</code> , POST <code>.../:id/restore</code>	Login
Schools	same pattern at <code>/schools</code>	Login
Communities	same pattern at <code>/communities</code>	Login
Cleanup events	<code>/cleanup/events</code> (CRUD + restore), POST <code>.../:id/attendance</code> , GET/POST <code>.../:id/waste-records</code> , GET <code>/cleanup/events/recent-waste-records</code>	Login
Inventory	<code>/inventory</code> (CRUD + restore), GET <code>/inventory/low-stock</code> , POST <code>/inventory/:id/transactions</code>	Login
Clubs	same pattern at <code>/clubs</code>	Login
Reports	<code>/reports</code> (CRUD + restore), PATCH <code>/reports/:id/status</code>	Login
Donations	POST <code>/donations</code>	Public
	GET <code>/donations</code> , <code>/donations/stats</code> , <code>/donations/:id</code> , PATCH <code>/donations/:id/status</code>	Login
Newsletter	POST <code>/newsletter</code>	Public
	GET <code>/newsletter</code>	Login
Partnerships	POST <code>/partnerships</code>	Public
	GET <code>/partnerships</code> , <code>/partnerships/:id</code> , PATCH <code>/partnerships/:id/status</code>	Login
Analytics	GET <code>/analytics/waste-by-type</code> , <code>participants-by-status</code> , <code>cleanup-events-by-status</code> , <code>inventory-transactions-by-type</code> , <code>reports-by-status</code> , <code>donations-by-status</code>	Login

In development, interactive API docs are at `/api` on the backend.

9. Database

PostgreSQL, managed with Prisma. The schema is `backend/prisma/schema.prisma`; migrations are in `backend/prisma/migrations/` (committed to Git, applied in order).

Main tables

Table	Holds
<code>users</code>	Staff accounts (email, bcrypt password hash, role, active flag)
<code>communities</code> , <code>schools</code> , <code>participants</code>	People and places
<code>cleanup_events</code> , <code>cleanup_attendances</code> , <code>waste_records</code>	Cleanup activity and waste collected
<code>inventory_items</code> , <code>inventory_transactions</code>	Supplies and stock movements
<code>environmental_clubs</code> , <code>weekly_reports</code>	Clubs and their weekly reports
<code>donors</code> , <code>donations</code>	Supporters and their pledges
<code>newsletter_subscribers</code>	Newsletter signups
<code>partnership_inquiries</code>	"Become a Partner" submissions

Soft deletes

Most tables have a `deleted_at` column. Deleting sets it instead of removing the row; every list, count and chart ignores rows where it is set. To permanently remove data you must do it in SQL.

Changing the schema

```
# development only
npx prisma migrate dev --name describe_the_change
# production
npx prisma migrate deploy
```

Commit the new folder in `prisma/migrations/`. After schema changes run `npx prisma generate` (part of `npm run build`).

Backups (do this)

Enable automatic backups at your database provider. To take one yourself:

```
pg_dump "$DATABASE_URL" -Fc -f eco-girls-$(date +%F).dump
```

Restore into an empty database:

```
pg_restore --clean --no-owner -d "$DATABASE_URL" eco-girls-YYYY-MM-DD.dump
```

Store backups somewhere other than the server that hosts the database, and test a restore before you need one. The dashboard does **not** have a working backup button (an earlier version had a fake one, which was removed).

10. Testing and verification

Quality checks

```
# backend
cd backend && npx tsc --noEmit && npx eslint "src/**/*.ts" && npm run build
# frontend
cd frontend && npx eslint app lib components && npm run build
```

The smoke test

`backend/scripts/smoke-test.mjs` checks the security model of a **running** backend with 64 checks: what is public, what needs login, forged tokens, form validation, login, change password (and restore), roles (manager blocked from user management), removed users losing access immediately, and that deleted records disappear from counts.

```
cd backend
API_URL=http://localhost:4000 \
ADMIN_EMAIL=admin@ecogirlscollective.org \
ADMIN_PASSWORD='your-admin-password' \
node scripts/smoke-test.mjs
```

- Start the backend with `LOGIN_RATE_LIMIT=1000` for this run, otherwise the test's own logins hit the rate limit.
- It creates records named `SMOKE-TEST...` / `smoke-...@example.com`. It hides its temporary manager and community with a normal delete, but the rows stay in the database, along with the newsletter, partnership and donation test entries. Purge them all with SQL: `DELETE FROM newsletter_subscribers WHERE email LIKE 'smoke-%'; DELETE FROM partnership_inquiries WHERE organization_name = 'SMOKE-TEST Org'; DELETE FROM donations WHERE donor_id IN (SELECT id FROM donors WHERE email LIKE 'smoke-%'); DELETE FROM donors WHERE email LIKE 'smoke-%'; DELETE FROM users WHERE email LIKE 'smoke-manager-%'; DELETE FROM communities WHERE name LIKE 'SMOKE-TEST Community%';`
- It temporarily changes the admin password and changes it back.
- Prefer running it on a staging copy. Expected last line: `Result: 64 passed, 0 failed`.

Run it after every deploy that touches the backend, and any time you change which routes are public.

Verified at handover

Backend and frontend production builds, lint and type checks all pass; the smoke test passes 64/64 against the production build; the browser flows (public site data, login redirect, dashboard pages, settings) were exercised in a real browser with no console errors.

11. Troubleshooting

Start with: **which side is failing?** Open the browser's developer tools (F12) → **Network** tab and look at the failing request's status code.

Symptom	Likely cause	Fix
Backend exits at startup with <i>"JWT_SECRET is missing, the placeholder value, or shorter than 32 characters"</i>	Production safety check	Set a random 32+ character <code>JWT_SECRET</code>
Seed fails with <i>"SEED_ADMIN_PASSWORD must be set..."</i>	First admin in production needs a password	Set <code>SEED_ADMIN_PASSWORD</code> for that run
Can't sign in ("Invalid credentials")	Wrong password, user removed, or user inactive	Have an admin check Settings → Team members ; reset via the seed (below) if it is the only admin
"Too Many Requests" on login	More than <code>LOGIN_RATE_LIMIT</code> attempts in a minute	Wait a minute. If it happens to everyone constantly, the API is behind a proxy: set <code>TRUST_PROXY=1</code>
Dashboard keeps returning to the login page	Token expired, or <code>JWT_SECRET</code> was changed (signs everyone out)	Sign in again
Forgot the only admin's password		On the backend: <code>SEED_ADMIN_EMAIL=you@org SEED_ADMIN_PASSWORD='new-password' npm run seed</code> . This resets that admin's password (and re-activates the account).
Public site shows <code>–</code> for numbers, empty "Where we work", missing charts	Website cannot reach the API	Check <code>NEXT_PUBLIC_API_URL</code> (rebuild after changing), that the API is running, and the Network tab
Browser console: CORS error	<code>FRONTEND_URL</code> does not match the site's address	Must match exactly: scheme + host (+ port), no trailing slash; <code>www.</code> and non- <code>www.</code> are different
Forms say "Could not send / subscribe"	API unreachable or validation error	Network tab: <code>400</code> = bad input (message shows which field), <code>5xx</code> = check backend logs
Dashboard pages show <i>"Failed to load ..."</i>	Session expired (401) or API down	Sign in again; check the API
Cannot find module <code>'.../generated/prisma'</code> or missing model errors	Database client not generated (<code>generated/</code> is git-ignored)	<code>npx prisma generate</code> (also run by <code>npm run build</code>)
Migration errors on deploy	Database unreachable or a migration conflicts	Check <code>DATABASE_URL</code> ; <code>npx prisma migrate status</code> shows what is applied
<code>EADDRINUSE</code> / <i>"Port 3000 is in use"</i>	An old server is still running	Stop the old process. Windows: <code>Get-NetTCPConnection -LocalPort 3000 % { Stop-Process -Id \$_.OwningProcess -Force }</code>
<code>/api</code> returns 404 in production	Swagger is intentionally hidden	Set <code>ENABLE_SWAGGER=true</code> if you really need it

Symptom	Likely cause	Fix
Donation stays "Pending" forever	No payment provider is connected	Expected; see limitations
Git shows "LF will be replaced by CRLF" warnings	Windows line-ending setting	Harmless

Where to look

- **Backend logs:** the process output of `npm run start:prod` (or your host's log viewer). Each request error and startup message appears here.
- **What the browser sent/received:** developer tools → Network.
- **Database:** connect with `psql "$DATABASE_URL"` and inspect tables directly.

Debugging a security question

Reproduce it with the smoke test's approach: request the route with no token (expect `401`), with a valid token (expect `200`), and, for admin-only routes, with a manager's token (expect `403`). To make a route public, add `@Public()` to its controller method; to protect it, remove it. Update the table in [Section 5](#) and re-run the smoke test.

12. Known limitations and next steps

Area	State	Suggested next step
Payments	The donate page records a pledge only. No card, mobile-money or bank payment is processed.	Connect a payment provider (for example Stripe, Flutterwave, or a local mobile-money gateway). The <code>Donation</code> table and status workflow are ready; you need provider credentials and a webhook route to mark donations <i>Completed</i> .
Email	Nothing sends email. Newsletter and inquiries are lists staff read in the dashboard.	Connect an email service, or export the newsletter list into your existing tool.
Content	Story quotes, story descriptions and some photos are placeholders.	See Section 7 .
Restore deleted records	API only, no button.	Add a "recently deleted" view.
Session storage	Login token lives in <code>localStorage</code> .	Move to an HTTP-only cookie if you later add user-generated content or more third-party scripts.
Password changes	Do not sign out other devices.	Add a token version to the user record.
Automated tests	The smoke test covers security and behaviour end to end; there are no unit tests.	Add tests around donations and reports as they gain logic.
Dependency warnings	<code>npm audit</code> lists 4 "high" items, all inside the Prisma command-line tool (<code>mysql2</code> , <code>deepmerge-ts</code>), which this PostgreSQL project does not use at runtime. The only offered fix downgrades Prisma to an older major version, so it was not applied.	Re-check <code>npm audit</code> when upgrading Prisma.
Monitoring	None.	Add uptime monitoring on the API root (<code>GET /</code>) and error alerts.
Manual figures	Some landing page numbers (<code>80+</code> girls, <code>3+</code> schools) are typed in.	Replace with live values from <code>/public/summary</code> once you trust the data.